Build on Ethereum.

Six carefully scoped challenges, progressing from your first smart contract to a complete Web3 application.

THE FORMAT

4 difficulty levels | 6 problem statements | Solidity + dApps

CHALLENGE MAP

LEVEL 01 Foundations

Student Records • ERC-20 Cryptocurrency

LEVEL 02 Contract Logic

Auction System • NFT Certificates

LEVEL 03 Full-Stack Web3

Campus Lost & Found dApp

LEVEL 04 Integrated dApp

Habit Tracker + NFT Achievement Badges

Student note: All financial operations must use local or testnet assets only. A frontend is required for Levels 3 and 4; Levels 1 and 2 can be demonstrated through Remix or contract tests.

LEVEL 01 PS 01 Selected from Level 1 / Option 2

Student Record Management

Create an owner-managed student database on Ethereum.

FORMAT	CORE SKILLS
Smart contract only	Storage, mappings, permissions

THE CHALLENGE

A college wants a transparent way to verify sample student records without relying on a central database. Build a Solidity contract that allows an authorized administrator to create and update records, while permitting public read-only verification.

MANDATORY REQUIREMENTS

01 Define a Student record containing a roll number, name, marks (0-100), and an existence flag.

02 Allow only the contract owner (administrator) to add new student records.

03 Reject duplicate roll numbers and prevent updates to records that do not exist.

04 Allow the owner to update existing names and marks without changing the roll number.

05 Provide a public read function that returns a stored student record by roll number.

06 Reject unauthorized updates and invalid marks with clear transaction reverts.

TECHNICAL GUIDELINES

Use Solidity and Remix IDE. A frontend, database, and server are not required. Use fictional student names and marks only; on-chain data is public.

ACCEPTANCE TESTS / DEMONSTRATION

1. Add a record and retrieve it successfully.
2. Update marks from the owner account.
3. Demonstrate that a different wallet cannot create or edit a record.
4. Show that duplicate records and out-of-range marks are rejected.

OPTIONAL STRETCH GOAL

Emit events whenever a student is created or updated.

Create Your Own Cryptocurrency

Deploy a simple, standards-based ERC-20 token.

FORMAT	CORE SKILLS
Smart contract only	ERC-20, balances, token transfers

THE CHALLENGE

Imagine a digital token used as points within a college club. Create a fungible token that users can hold and send to one another, using a standard Ethereum token implementation rather than writing every token operation from scratch.

MANDATORY REQUIREMENTS

01 Create an ERC-20 token with a meaningful name and symbol.

02 Set an initial supply of 1,000,000 tokens and assign it to the deployer at deployment.

03 Use OpenZeppelin ERC-20 to support standard balanceOf, totalSupply, and transfer behavior.

04 Let any token holder transfer tokens to another Ethereum address.

05 Demonstrate transfers between at least two different wallets.

06 Verify that transfers fail when the sender lacks sufficient tokens.

TECHNICAL GUIDELINES

Use Solidity, Remix IDE, and OpenZeppelin Contracts. This is an educational test token, not a public investment or exchange listing. A frontend is optional.

ACCEPTANCE TESTS / DEMONSTRATION

1. Show initial supply and deployer balance.

2. Transfer tokens from wallet A to wallet B.

3. Verify A decreases and B increases by the transferred amount.

4. Attempt a transfer larger than the sender balance and show it fails.

OPTIONAL STRETCH GOAL

Add a clearly documented owner-controlled minting function or a token-burning feature.

LEVEL 02 PS 03 Selected from Level 2 / Option 2

Blockchain Auction System

Run a time-limited auction with safe refunds.

FORMAT	CORE SKILLS
Smart contract only	Payable bids, deadlines, withdrawal safety

THE CHALLENGE

A seller wants to auction a digital collectible for test ETH. All bids must be transparent, the contract must remember the highest valid bid, and bidders who lose must be able to reclaim their funds.

MANDATORY REQUIREMENTS

01 Let an auction owner set an item description, minimum bid, and auction deadline.

02 Accept bids in test ETH only while the auction is active; each valid bid must exceed the current highest bid.

03 Track the highest bidder and highest bid publicly.

04 Credit displaced bidders with withdrawable refunds; do not automatically send refunds inside the bidding function.

05 Prevent bidding after the deadline and allow the auction to be finalized only once.

06 After finalization, allow the seller to withdraw the winning amount; handle the no-bid case.

07 Emit events for valid bids, refunds, and finalization.

TECHNICAL GUIDELINES

Use Solidity and Remix or Hardhat. Follow the checks-effects-interactions pattern in withdraw functions. No real-world delivery or physical asset transfer is required.

ACCEPTANCE TESTS / DEMONSTRATION

1. Place a valid first bid and a higher second bid from another wallet.
2. Reject a low bid and a bid placed after the deadline.
3. Let an outbid wallet claim its refund exactly once.
4. Finalize the auction and withdraw the winning amount only once.

OPTIONAL STRETCH GOAL

Add a feature to cancel the auction before any bids have been placed.

LEVEL 02 PS 04 Selected from Level 2 / Option 3

NFT Achievement Certificates

Issue verifiable, non-transferable student credentials.

FORMAT	CORE SKILLS
Smart contract only	ERC-721, minting, issuer permissions

THE CHALLENGE

Build an on-chain certificate system for courses, workshops, or hackathons. Authorized issuers should be able to award unique achievement NFTs to students, and anyone should be able to check who received a certificate.

MANDATORY REQUIREMENTS

01 Create an NFT-based certificate contract with a unique certificate/token ID.

02 Store or reference a certificate title, issuer, recipient wallet, and issuance information.

03 Allow only an authorized issuer to mint certificates to student wallets.

04 Provide a public method to verify whether a certificate exists and identify its recipient.

05 Prevent the same certificate ID from being issued twice.

06 Make certificates non-transferable after issuance (soulbound behavior).

07 Emit an event when a certificate is issued.

TECHNICAL GUIDELINES

Use Solidity and an OpenZeppelin ERC-721 implementation or an equivalent documented NFT standard. Non-transferability must be enforced by the contract, not just hidden in the UI.

ACCEPTANCE TESTS / DEMONSTRATION

1. Issue a certificate to a student wallet and verify ownership.
2. Reject an issuance attempt from an unauthorized wallet.
3. Reject a second mint using an existing certificate ID.
4. Demonstrate that transferring an issued certificate is blocked.

OPTIONAL STRETCH GOAL

Support revocation by the issuer and expose certificate validity in the verification method.

LEVEL 03 PS 05 Selected from Level 3 / Option 1

Decentralized Campus Lost & Found

A wallet-connected application for lost and found items.

FORMAT	CORE SKILLS
Full Web3 application	Frontend, MetaMask, contract integration

THE CHALLENGE

Students frequently misplace belongings on campus. Build a decentralized application that lets students publish lost or found item listings, browse posts, and mark their own listings as resolved. Listing ownership must be tied to an Ethereum wallet.

MANDATORY REQUIREMENTS

01 Build a frontend that connects to a user wallet (for example, MetaMask) and displays the connected address.

02 Create a smart contract to store listings with an ID, title, category, lost/found type, approximate location, creator address, and status.

03 Allow connected users to create new listings through the frontend.

04 Display active listings in a usable feed and provide at least one filter (type, category, or status).

05 Allow only the creator of a listing to mark it as resolved or closed.

06 Provide a My Listings view for the connected wallet.

07 Show helpful UI feedback for pending, successful, and failed transactions.

TECHNICAL GUIDELINES

Use Solidity, HTML/CSS/JavaScript or React, and ethers.js (or an equivalent Web3 library). A traditional backend is not required. Avoid putting personal phone numbers or private contact details on-chain.

ACCEPTANCE TESTS / DEMONSTRATION

1. Connect a wallet and publish a lost item.
2. Publish a found item from another wallet.
3. Filter and open both listings in the interface.
4. Resolve a listing as its creator and reject the same action from a different wallet.

OPTIONAL STRETCH GOAL

Support optional image links stored off-chain and shown within each listing.

LEVEL 04 PS 06 Combined from Level 3 / Options 4 + 5

Habit-to-Badge Web3 dApp

An on-chain habit tracker that rewards progress with NFT badges.

FORMAT	CORE SKILLS
Integrated full Web3 application	Habit tracking, milestone logic, NFTs, UI

THE CHALLENGE

Combine a personal habit-tracking application with an NFT badge collection. Students connect a wallet, create habits such as reading or coding, record completions, and unlock unique achievement badges when they reach milestones. Achievement eligibility must be verified by smart-contract logic.

MANDATORY REQUIREMENTS

01 Build a wallet-connected frontend with a habit dashboard and personal NFT badge gallery.

02 Allow a user to create, view, and deactivate personal habits; each habit belongs to its creator wallet.

03 Allow the owner to mark a habit complete no more than once per UTC calendar day, based on the on-chain timestamp.

04 Track per-user total valid check-ins and display progress in the frontend.

05 Implement at least two NFT milestones: First Step (1 valid check-in) and Consistency Starter (5 valid check-ins).

06 Allow each eligible wallet to claim each achievement NFT at most once; ineligible claims must revert.

07 Display earned badges and their milestone descriptions, with badge ownership verifiable on-chain.

TECHNICAL GUIDELINES

Use Solidity, ERC-721, React or vanilla JavaScript, wallet integration, and ethers.js or equivalent. Habit entries are self-reported; no external truth verification is required. For the 5-check-in demo, students may complete five distinct habits on the same day.

ACCEPTANCE TESTS / DEMONSTRATION

1. Create two habits and record a daily check-in.
2. Reject a repeated same-day check-in for the same habit.
3. Claim the first badge and reject a duplicate claim.
4. Reach five valid check-ins, claim the second badge, and show both in the gallery.
5. Switch wallets to verify that habits and badges remain wallet-specific.

OPTIONAL STRETCH GOAL

Show a 7-day streak calculated from valid UTC check-in days, and provide automated tests using simulated blockchain time.

SUBMISSION & EVALUATION

What to submit

The requirements below apply to every chosen problem statement. The specific mandatory requirements on each project page are the main evaluation checklist.

REQUIRED SUBMISSION PACKAGE

01 Source code: A GitHub repository or ZIP containing all Solidity files and, for Levels 3-4, the frontend source.

02 README: A short description, technology stack, setup steps, compiler version, network used, and instructions to test the contract/app.

03 Deployment details: Provide the deployed contract address and network where applicable; for local-only work, include the deployment procedure.

04 Testing evidence: Screenshots, console output, or test logs demonstrating both successful and failing cases listed under Acceptance Tests.

05 Demo: A short screen recording or live walkthrough showing the main functionality (including wallet interaction for Levels 3-4).

COMMON RULES

- Use test ETH and local/test networks only. Never request real funds or private keys.
- Do not hard-code user wallet addresses for ownership checks; use transaction-sender logic.
- Code must be written and explainable by the team. Third-party libraries may be used if identified in the README.
- Reject invalid operations in the smart contract itself, not only through frontend validation.
- For Levels 1-2, a working frontend is not required; for Levels 3-4, it is required.

SUGGESTED EVALUATION RUBRIC

Criterion	Weight	What reviewers look for
Functional requirements	40%	Mandatory features implemented and working
Contract correctness / security	20%	Access checks, validation, safe asset handling
Code quality	15%	Clear structure, naming, reuse, readability
Testing	15%	Success paths and failure cases demonstrated
Documentation / demo	10%	Reproducible setup and clear explanation

IMPORTANT FOR PARTICIPANTS

The stretch goal is optional and should not replace or delay the mandatory requirements. Reviewers should prioritize a correct, complete core implementation over additional features.